

Introduction To Cryptography

Year 11 Profile Course with Ms. Fernandez and Josh

November 15, 2017

1 Getting Started

Whenever one starts a mathematical adventure it helps to have a common vocabulary, a set of ideas or models that we have in our head when we want to make statements about a given problem or example.¹ In the realm of functions, traditional mathematical terminology is sometimes not particularly evocative,² so we will begin with some notation and general concepts and conventions so we can be more exact (but also clearer) when we tackle some interesting questions later on. In fact, it will then be up to you to define your own additional notation to more easily and clearly solve a given set of problems (and hopefully pose a few questions yourself). In addition to the next section, there is also an appendix—this describes a couple more in depth examples and gives you more material to help digest the new concepts below. If you need help with language or struggle with a given concept, your first and most powerful tool is your own brain! Give yourself time! Mark this paper up! Write questions in the margins! (There is a digital version in Dropbox anyway) Skip ahead! Backtrack! If you still feel lost, go ahead and ask a friend who doesn't look like they mind answering, or ask us, your guides on this learning adventure. Speaking of adventure, let's get this one started!

1.1 The Quick and Dirty Summary

Alphabet, Words, and Languages

We start with the basics: an *Alphabet* is just a list of characters—these can be letters or numbers or just random nonsense symbols—it doesn't really matter! The point is we combine these symbols to form... wait for it... *words*. Kind of like the plot of a soap opera, we don't assign meaning to these collections of characters, we are just concerned with their broad patterns and structure. For example, we might have a language where our alphabet is $A = \{a, b, c, d, e\}$ and the associated language (or the language “induced by” that alphabet) is all the words³ we can create with this alphabet. We will write as this language

¹We are essentially seeking to create “shared representations” by which we can broaden our ways of seeing the world.

²At least in English, in German the words are way better!

³of finite length, we don't want infinite words here... that might come later... ;)

(associated with A) as $\mathcal{L}(A)$. This means that $abcabcabc$ is in $\mathcal{L}(A)$ but abf is not—that “ f ” symbol isn’t a thing in our alphabet so we can’t use it here!

A Bit of Interpretation and Some Operations

As another example, we might think about all words made from the much more limited alphabet $B = \{0, 1\}$. But interestingly enough $\mathcal{L}(B)$ includes a binary representation of *all whole numbers* but also infinitely many more! Notice how we get all the normal binary numbers (pause and ponder here so you can actually notice that... don’t worry I’ll wait, I’ve got time...) but also we find words like 0001 and 00111110101001 of the form $(0^n) + (\text{somebinarynumber})$: where we have some amount of zeros on the front of our word. But wait a minute... this is not a number yet—we can interpret these symbols that way because we are clever humans and not computers⁴ but we are just dealing with 1 and 0 as symbols now. Note that I used a superscript $(\cdot^n)$ to reference a situation where a character is repeated. Some people use the multiplication symbol as well: $abcd * 2 = abcdabcd$, for example. You can use whatever you feel like, as long as you are clear and at least moderately consistent.

Speaking of operations and their symbols, addition means to “append” so $abcd + abcd = abcd * 2 = abcdabcd$, and $aaa + bbb = aaabbbb$, it’s a simple thing, but quite useful.⁵ Note that addition works this way even when our symbols are numbers: so $101 + 10$ does *not* equal 111, but rather 10110. If you find yourself in a situation where you want to use both numerical addition and symbolic append, then just write $append(w_1, w_2) = w_1w_2$ All clear? Great!

Functions and Spaces

As this batch of lessons is really about functions, we will be dealing with the buggers a lot! When we deal with encryption, we will specify two key functions: encrypt and decrypt, each of which take a word as an *argument* (or *input*) and a word as an *output*. We will write these functions as $encrypt(\cdot)$ and $decrypt(\cdot)$ to show that they are functions and not words or terms. The “ $\cdot$ ” means “you can put an argument here!” where in this case the argument is a word.⁶ For some other functions we might have $cleverfunctionname(\cdot, \cdot)$ indicating this particular function can take takes two arguments (or two inputs) but usually only one output. Generally when it comes to functions we only have *one* output. Of course there are exceptions, the world wouldn’t be as interesting without them. As one could guess, there are also functions with more than two arguments and we highly recommend you get used to functional notation and naming (which

⁴Just a joke future robot overlords, at the time of this writing computers were significantly less glorious than your current incarnations.

⁵If you look at our definition of addition it is no wonder that some people use multiplication for a “multiple append” rather than exponentiation—but let’s not get so religious about notation—as long as it is clear and consistent it works for us!

⁶This leads to the idea of “types” in both mathematics and computer science—well worth a Wikipedia search!

is relatively standardized, though we will use longer names for functions so that they are easier to remember).

In the case of $encrypt(\cdot)$ and $decrypt(\cdot)$ we have two languages to worry about (not to mention the words that make them up!): We call a word *before* it are encrypted is *plaintext* and after it is encrypted it is *ciphertext*—that means we, sensibly, decrypt ciphertext into plaintext. Here is a slightly more mathematical way of doing it (with more notation):

$$\begin{array}{ccc} \text{plaintext} & \xrightarrow{encrypt(\cdot)} & \text{ciphertext} \\ \text{ciphertext} & \xrightarrow{decrypt(\cdot)} & \text{plaintext} \end{array}$$

In talking about plaintext and ciphertext we usually talk about “spaces” – a fancy mathematical term that sounds cooler than it is, unfortunately. A space is really just a collection of all possible possibilities of something, the set of possible outcomes of something in probability is an “event space,” all possible distances associated with a means of measuring (called a metric) is called a metric space. So in this case the space of all possible words that could be our plaintext and the space of all possible words that could be our ciphertext form *spaces* of their own. We call these the message space and the coded space respectively (which makes sense right?).⁷ In general when we deal with functions we talk about the *domain* and *range* of a function. Also known as the function’s *preimage* and *image*. In this case, our *domain* is the space of all possible plaintext (our message space) and our *range* is the space of all possible cipher text (our coded space). *These are all just terms for the same thing!* In general, the ideas of domain and range are super important when it comes to describing how functions behave and their effects: they tell us what our function can “act on” and where it can “send things to” – the German terms for what functions do and how they do them is particularly visual, and I highly recommend looking it up to give yourself a better idea of what is going on. These are abstract concepts that were developed relatively late on our human mathematical time-line⁸ We will cement and make these terms and their meanings more clear in the coming weeks—the ideas are not that complex, but their implications can be.

Transition

The next section goes into some of these ideas in more detail with a few examples, but if you want to jump right into some cool problems and some basic codes and encryption—jump ahead to the next section on how to pass notes in class. The ideas are all relatively small, but they will build up fast, and the challenge for most of this stuff is being clear about what we are talking about and in what space we happen to be. Have fun!

⁷If it doesn’t just ask Schlomo, and even if it does try drawing a picture of these spaces for different encryption schemes as they come up! Always a good idea!

⁸only in the last 200 years, give or take—it is hard to tell with such things, as forms of related principles existed a bit before that.

2 How to Send Messages in Class

Before the age of instant messaging on those newfangled, brain destroying, zombifying devices that you kids have these days, students passed notes in class on old-fashioned paper with messages written with old-fashioned ink. If one was clever (and *very* bored) one might invent a shift cipher—where each letter is shifted in its normal order by a certain amount so that no one other than those who knew the key (or shift amount) could read it (or that it would take long enough to decode that no one else would care to try...). Let’s start with a smaller alphabet, always a good strategy to begin—start small: our smaller alphabet is $A = \{a, b, c, d, e\}$ where a shift of three would produce the cipher-alphabet $A' = \{d, e, a, b, c\}$. Let’s look at this more visually with the full English Alphabet coupled with first a shift of 1 and then a shift of 8:

Shift 1:

$encrypt(abcdefghijklmnopqrstuvwxyz) \rightarrow bcdefghijklmnopqrstuvwxyz a$

Shift 8:

$encrypt(abcdefghijklmnopqrstuvwxyz) \rightarrow ijklmnopqrstuvwxyz abcdefgh$

One interpretation of this is that we are first mapping each letter in our alphabet to a number ($a \rightarrow 1, b \rightarrow 2$ and so on up to $z \rightarrow 26$ —note that I chose to start with 1 but you don’t have to!) then, with that mapping in mind, we add our shift amount to that number (in the second example 8). When we go over the number of characters in our alphabet we just subtract that number (in this case 26) and start again!⁹ With all this in mind, how does the *decrypt*($\cdot$) function work? (or how should it work so that it does what we expect? Pause and ponder for a moment...) After you’ve thought about that for a while, and are happy with your (hopefully nicely written) answer, let’s move on to the main event—problems! Successfully resisted urge to make a 99 problems pun!

Shift Cipher Problems

These problems assume that we have a shift cipher on our normal English alphabet (sorry German). Let’s start things slow and then get to juicier questions for our second question:

1. If our *key* (shift amount) is 6:

- (a) what are $encrypt(a), encrypt(j), encrypt(y)$?
- (b) What would the encrypted version of “albatross” and “sequester” look like?
- (c) What are $decrypt(abcd)$ and $decrypt(YOUR_NAME)$

⁹This is called Modular Arithmetic, something that is sometimes scarily useful, but we don’t really have it in school—not quite sure why!

- (d) Is there a word that is made of up of all the same letters as its encrypted counterpart? How long can it be?
 - (e) For all the words you encrypted in the previous questions, see if you can encrypt them again with 20 as your key, what happens? What pattern do you notice? What would the equivalent be if our key was 17, or if it was 13? (you don't necessarily need to do all of these over again of course, just demonstrate that you understand what is going on.) How might we write the relationship between a 6-shift code and a 20 shift code?
2. Now let's switch to a more general key k , so we don't specifically how much we are shifting by, perhaps we can still say some cool stuff about these ciphers—feel free to use examples:
- (a) A *palindrome* is a word that is the same backwards and forwards (like mom, kayak, racecar, or deified), are palindromes still palindromes after we encrypt them with a shift cipher? Can you prove it? What if we use some other type of cipher on the same alphabet? What notation might you use to prove it with fewer words or symbols? (Be sure to clearly define what you mean by each symbol or abbreviation!)
 - (b) In order to do the previous problem, you probably had to invent some type of notation that conveys the reverse of a given word: how would you write a function $reverse(\cdot)$ in terms of the letters in a given word $w = w_0w_1 \cdots w_{n-1}w_n$? What happens if we do $reverse(reverse(w))$? With that in mind, what can you say about the inverse of $reverse(\cdot)$?
 - (c) If I give you access to an encryption machine, and tell you that it is a Shift Cipher, how many times must you use the machine in order to guess which shift I am using? What method would you use? Suppose I change the alphabet of the machine, what would you need besides just the new symbols I am using in order to figure out my shift cipher in the same amount of steps?
 - (d) Schlomo has solved the previous problem and is rather unhappy with his result, he wants to make a given Shift Cipher more “secure” by shifting each letter twice, first by k_1 then again by k_2 . Does this make the cipher more secure? Why or why not? As a hint, it might be helpful to think of $encrypt(\cdot)$ as a *multi-input function*, that is, a function that takes in more than one input (just like it says on the tin!). So instead of $encrypt(w)$ with an “implicit” key k for a given word w , we can try something like $encrypt(w, k)$. How might one write Schlomo's strategy with this new(ish) function? Can you think of another strategy involving manipulating keys that makes this “more secure” (in your answer, be sure to specify what you mean by more secure).¹⁰

¹⁰by the way, what we just did is to “uncurry” our encrypt function: currying is where we use multiple one-argument/input functions to describe a multi-argument/input function,

uncurrying is the reverse! Can you think of other examples where this might be useful? If you are interested in this, bug us about it! (But also look it up it's super cool!)